Compresión de imágenes con Quadtree mediante Divide y Vencerás

Reporte técnico — Diseño, análisis teórico y evaluación experimental

Curso: _____
Equipo / integrantes: _____
Fecha: _____

A. Modelado del problema

Entrada. Una imagen $I \in \{0,\ldots,255\}^{H\times W\times C}$ (C = 3 para RGB o C = 1 para grises), cargada desde archivo y guardada como matriz *uint8*, más una tolerancia $\tau \geq 0$ y un criterio de uniformidad. El tamaño de entrada es N = H·W píxeles; para imágenes cuadradas, n = H = W y $N = n^2$. **Salida.** (i) Un árbol Quadtree T; (ii) su codificación binaria; (iii) la imagen reconstruida Î con las mismas dimensiones; (iv) métricas (nodos, hojas, profundidad, bytes, razón de compresión CR, MSE, PSNR, error máximo).

Similitud de color. Una región R es *uniforme* si $\mathbf{\max_c (\max_R I_c - \min_R I_c) \leq \tau}$ (criterio «range», norma L_∞ por canal). Se eligió porque da una **cota garantizada** del error: el color representativo es la media redondeada de R, que cae dentro de [min, max]; por lo tanto $|I_c(p) - \hat{I}_c(p)| \leq \tau$ para todo píxel p y canal c, y $MSE \leq \tau^2$. Con τ = 0 la región debe ser de color exactamente constante y la compresión es **sin pérdida**. Como criterio alternativo (opcional) se implementó «var»: $\mathrm{Var}_R(I_c) \leq \tau^2$ para todo c, evaluable en Θ(1) con imágenes integrales [5]; en aritmética entera exacta la condición es $N_R \cdot \Sigma I^2 - (\Sigma I)^2 \leq \tau^2 \cdot N_R{}^2$.

Almacenamiento y métricas. Formato binario propio: cabecera de 48 bits (marca, C, alto, ancho) y recorrido en preorden con 1 bit por nodo (1 = hoja, 0 = interno) más 8·C bits de color por hoja. No se guardan coordenadas ni punteros: el decodificador re-deriva las regiones con la misma regla de partición. Tamaño = 48 + nodos + 8·C·hojas bits (redondeado a bytes; verificado contra los bytes reales escritos). Razón de compresión **CR = 8·C·H·W / bits_comprimidos** (CR > 1 comprime; CR < 1 expande). MSE = media de $(I - \hat{I})^2$ sobre todos los píxeles y canales; $PSNR = 10 \cdot \log_{10}(255^2/MSE)$.

B. Diseño algorítmico

Estrategia. *Dividir*: si la región no es uniforme y tiene más de un píxel, se parte en sus cuadrantes. *Resolver*: se procesa recursivamente cada cuadrante; casos base: región uniforme (hoja con su color medio) o región de 1×1 (siempre uniforme). *Combinar*: los hijos se enlazan a un nodo padre que conserva la región (y, x, h, w), de modo que el árbol conserva la distribución espacial y basta recorrerlo para reconstruir la imagen.

```
CONSTRUIR(I, y, x, h, w, τ):
    si h·w = 1  ó  UNIFORME(I, y, x, h, w, τ):            # casos base
        devolver HOJA(y, x, h, w, color = redondear(media(I[región])))
    para cada (y', x', h', w') en PARTIR(y, x, h, w):      # DIVIDIR (2 ó 4 subregiones no vacías)
        hijos.añadir( CONSTRUIR(I, y', x', h', w', τ) )    # RESOLVER
    devolver NODO(y, x, h, w, hijos)                       # COMBINAR
```

Dimensiones arbitrarias. PARTIR usa $h_1 = \lfloor h/2 \rfloor$, $h_2 = h - h_1$ (igual para el ancho) y descarta los cuadrantes con alto o ancho 0 (solo ocurre si h = 1 o w = 1; entonces el nodo tiene 2 hijos). Así los cuadrantes cubren la región sin traslapes ni huecos y cada hijo tiene estrictamente menos píxeles que el padre: no se subdividen regiones vacías y la recursión siempre termina. Se probó con 100×75, 127×129, 1×20 y 20×1. **Estructura.** *Node* = (y, x, h, w, color | None, hijos[0..4]); una hoja tiene color y no tiene hijos. No se usó ninguna biblioteca que construya o comprima Quadtrees (NumPy solo maneja matrices; Pillow solo lee/escribe archivos). **Reconstrucción.** Recorrido recursivo que pinta el rectángulo de cada hoja con su color.

C. Análisis de correctitud y complejidad

Correctitud. (1) *Terminación:* el número de píxeles de cada hijo es estrictamente menor que el del padre y es ≥ 1; con área 1 se alcanza el caso base. (2) *Teselación:* por inducción, si los cuadrantes de una región la cubren exactamente una vez (propiedad de PARTIR), las hojas del árbol cubren la imagen exactamente una vez; verificado por prueba automática. (3) *Cota de error:* en cada hoja uniforme el color medio está en [min, max], de modo que el error por píxel es ≤ τ; con τ = 0 la reconstrucción es idéntica (verificado en 3 tipos de imagen y 2 criterios). (4) *Codificación:* la serialización → deserialización devuelve el mismo árbol (prueba `tests.py`, 8 pruebas).

Complejidad temporal. Operación fundamental: lectura/comparación de un píxel en UNIFORME (y creación de un nodo). Para n×n (potencia de 2), con el criterio «range» la verificación cuesta $\Theta(n^2)$ en una región de lado n, y sea $k \in \{0,\ldots,4\}$ el número de cuadrantes que resultan no uniformes: $\mathbf{T(n) = k \cdot T(n/2) + \Theta(n^2)}$, $T(1) = \Theta(1)$.

- **Peor caso** (k = 4 en todos los niveles; p. ej. ruido con τ = 0): a = 4, b = 2, $f(n) = \Theta(n^2) = \Theta(n^{\log_2 4})$ → caso 2 del Teorema Maestro: $\mathbf{T(n) = \Theta(n^2 \log n)}$. Hay $\leq (4n^2 - 1)/3$ nodos (medido: 87 381 para n = 256), pero cada nivel re-examina todos los píxeles y hay log n niveles.
- **Mejor caso** (imagen uniforme): una sola verificación, sin recursión, $\mathbf{\Theta(n^2)}$ (hay que leer todos los píxeles al menos una vez).
- **Caso intermedio** (fronteras de longitud O(n), p. ej. «regions»): a profundidad d solo $O(2^d)$ regiones son no uniformes, con área total $O(n^2/2^d)$; la suma geométrica de las verificaciones es dominada por la raíz → $\mathbf{\Theta(n^2)}$ (equivale a k < 4 en la recurrencia, caso 3 del Teorema Maestro).
- **Criterio «var»** con imágenes integrales: la verificación es Θ(1) y el preproceso $\Theta(n^2)$: $T(n) = 4T(n/2) + \Theta(1) = \Theta(n^2)$ en el peor caso y $\Theta(n^2)$ por el preproceso en el mejor.
- El cálculo del color medio de las hojas suma Θ(Σ áreas de hojas) = $\Theta(n^2)$ en total; no altera el orden.

Complejidad espacial y costos por separado. El árbol ocupa Θ(#nodos): Θ(1) en el mejor caso y $\Theta(n^2)$ en el peor ($\leq 4n^2/3$ nodos). La pila de recursión llega a $\lceil \log_2 n \rceil + 1$ marcos: $\mathbf{O(\log n)}$ (profundidades medidas: 6, 7, 8 para n = 64, 128, 256). El criterio «var» añade $2\cdot(n+1)^2\cdot C$ enteros de 64 bits. Ver tabla:

Operación	Tiempo (peor)	Tiempo (mejor)	Espacio extra
Construir (range)	$\Theta(n^2 \log n)$	$\Theta(n^2)$	árbol $\Theta(n^2)$ + pila $O(\log n)$
Construir (var)	$\Theta(n^2)$	$\Theta(n^2)$	árbol + tablas integrales $\Theta(n^2)$
Recorrer (estadísticas)	$\Theta(\#\text{nodos}) = O(n^2)$	$\Theta(1)$	pila $O(\log n)$
Reconstruir	$\Theta(n^2 + \#\text{nodos}) = \Theta(n^2)$	$\Theta(n^2)$ (se escribe cada píxel)	imagen de salida $\Theta(n^2)$

D. Resultados experimentales

Diseño. 3 tamaños (64^2, 128^2, 256^2) × 3 tolerancias ($\tau = 0$ exacta, 16, 48) × 3 contenidos: *uniform* (un color), *regions* (4 regiones planas: 3 rectángulos y un círculo) y *detail* (degradado + círculo + línea fina + ruido gaussiano $\sigma = 6$). Imágenes sintéticas con semilla fija (reproducible); tiempo = mediana de 5 ejecuciones (Python 3, NumPy, un solo hilo). Todos los registros (también tiempos de recorrido y reconstrucción, bytes, PSNR, error máximo) están en `results/results.csv`.

	τ = 0				τ = 16				τ = 48			
Imagen	**Hojas**	**CR**	**MSE**	**ms**	**Hojas**	**CR**	**MSE**	**ms**	**Hojas**	**CR**	**MSE**	**ms**
uniform 64^2	1	1347	0.0	0.22	1	1347	0.0	0.21	1	1347	0.0	0.24
uniform 128^2	1	5387	0.0	0.80	1	5387	0.0	0.80	1	5387	0.0	0.79
uniform 256^2	1	21546	0.0	3.08	1	21546	0.0	3.04	1	21546	0.0	3.05
regions 64^2	268	14.38	0.0	3.33	268	14.38	0.0	3.38	268	14.38	0.0	3.32
regions 128^2	532	29.07	0.0	8.17	532	29.07	0.0	8.17	532	29.07	0.0	8.14
regions 256^2	1246	49.75	0.0	21.9	1246	49.75	0.0	22.9	1246	49.75	0.0	22.8
detail 64^2	4096	0.95	0.0	32.8	2767	1.40	8.0	29.5	400	9.66	45.1	7.80
detail 128^2	16384	0.95	0.0	174.5	10687	1.45	8.5	101.3	745	20.78	45.4	12.0
detail 256^2	65536	0.95	0.0	703.1	41314	1.50	9.0	575.0	1447	42.85	41.8	34.3

Tabla 1. Hojas, razón de compresión CR, MSE y tiempo de construcción (ms). Nodos = (4·hojas − 1)/3 (árbol 4-ario completo en potencias de 2); profundidad máxima = $\log_2 n$ en todos los casos salvo «uniform» (0). Detalle completo en el CSV.

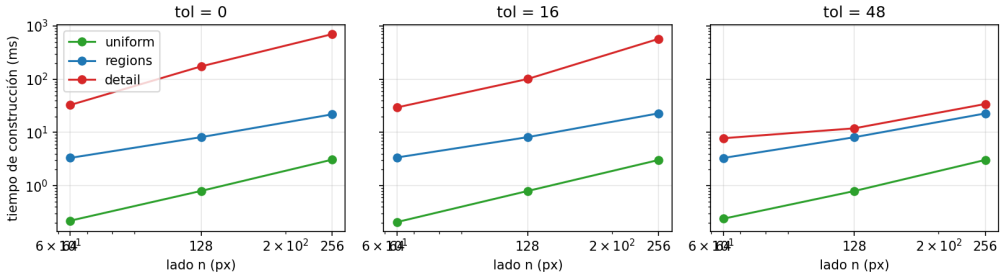


Figura 1. Tiempo de construcción vs. lado n (escala log-log) para cada tolerancia y tipo de contenido.

¿Cómo influye el tamaño? En «uniform» el tiempo crece ×3.6 y ×3.9 al duplicar n (≈ ×4 = $\Theta(n^2)$, una sola pasada). En «detail» con $\tau = 0$ (peor caso) crece ×5.3 y ×4.0; en «regions» solo ×2.5 y ×2.7, porque el número de nodos sigue el perímetro de las regiones ($\Theta(n)$): 357, 709, 1661 nodos. En la Fig. 3 (ruido, $\tau = 0$, n hasta 512) cada duplicación multiplica el tiempo por ≈ 4, consistente con $\Theta(n^2)$. *Nota honesta:* el factor log n del peor caso teórico no se distingue de $\Theta(n^2)$ en estas mediciones, porque en Python el costo por nodo (≈ $4n^2/3$ llamadas) domina sobre las operaciones vectorizadas de NumPy que ejecutan la verificación; la recurrencia describe el modelo de costo de lectura de píxeles, no el tiempo de pared de esta implementación.

¿Cómo afecta τ al número de regiones? En «detail» 256^2 las hojas pasan de 65536 ($\tau = 0$) a 41314 ($\tau = 16$) y 1447 ($\tau = 48$). El efecto es de umbral: mientras τ sea menor que la amplitud del ruido y del degradado en un bloque, solo se fusionan bloques diminutos; al superarla, bloques grandes se vuelven uniformes. En «uniform» y «regions» (colores planos) τ no cambia nada porque los bordes son nítidos. Con ruido leve ($\sigma = 2$) sobre las mismas regiones (Tabla 2), $\tau = 0$ y $\tau = 4$ apenas comprimen y la compresión se dispara cuando τ supera el rango del ruido (≈ 8–16).

τ	0	4	8	16	48
Hojas	65536	54178	12325	1255	1246
CR	0.95	1.15	5.04	49.40	49.75
MSE	0.00	0.38	3.21	4.03	4.03
Error máx.	0	3	6	9	9

Tabla 2. «regions» 256^2 con ruido gaussiano $\sigma = 2$ (experimento `noisy_regions` en extra_results.csv).

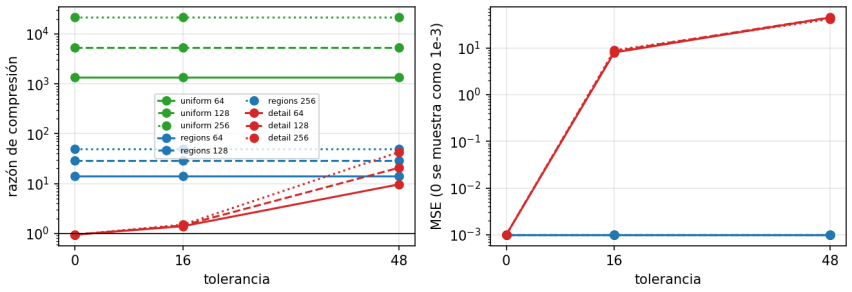


Figura 2. Razón de compresión (izq.) y MSE (der.) vs. tolerancia (MSE = 0 se dibuja como 10^{-3}).

Compresión vs. calidad. En «detail» 256^2: $\tau = 16$ da CR = 1.50 con MSE = 9.0 (PSNR = 38.6 dB) y $\tau = 48$ da CR = 42.9 con MSE = 41.8 (PSNR = 31.9 dB). El error máximo observado (12 y 30) respeta la cota τ. La relación no es lineal: el salto de CR ≈ 1.5 a ≈ 43 cuesta ≈ 7 dB de PSNR. **Casos sin ahorro:** con $\tau = 0$ y contenido ruidoso «detail» (y el ruido uniforme del peor caso) CR = 0.947 < 1: todas las hojas son píxeles, y el árbol añade 1 bit por nodo (4N/3 bits) a los 24N bits de datos, así que CR → 18/19 ≈ 0.947, es decir, **+5.6 % de almacenamiento**. También con $\tau = 16$ en «detail» el ahorro es mínimo (CR ≈ 1.4–1.5).

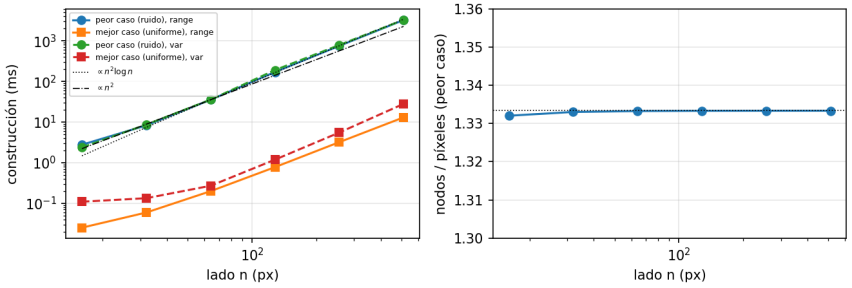


Figura 3. Izq.: escalamiento del tiempo (peor caso = ruido, $\tau = 0$; mejor caso = uniforme) con ambos criterios y referencias n^2 y $n^2 \log n$. Der.: nodos/píxeles en el peor caso → 4/3.

Criterio «var» (opcional). Con $\tau = 0$ construye exactamente el mismo árbol que «range». Con $\tau = 16$ en «detail» 256^2 tarda 17.8 ms frente a 503.0 ms, pero no son comparables a igual τ: «var» acota la varianza de la región, no el error por píxel (CR = 59.5, MSE = 80.5, error máx. = 133). **Dimensiones no potencia de 2** (100×75 y 127×129): todas las pruebas pasan y el comportamiento es análogo (p. ej. «regions» 127×129: CR = 24.7, MSE = 0).

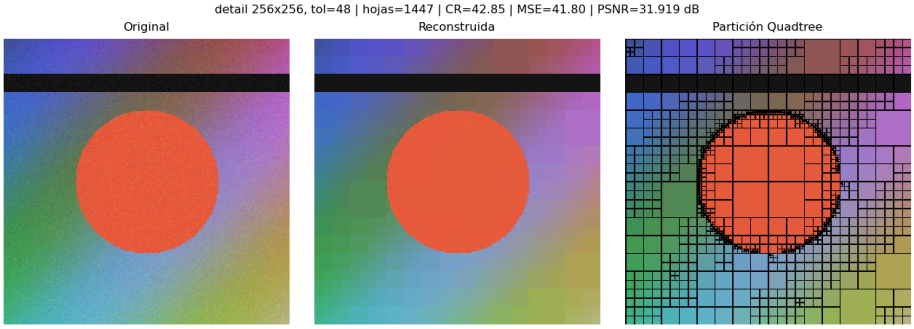


Figura 4. Evidencia (ejemplo 2): original, reconstruida y partición Quadtree. Los ejemplos 1 y 3 están en results/evidence/.

E. Conclusiones

- **Ventajas.** Algoritmo simple y correcto (cota de error ≤ τ demostrada y comprobada); con $\tau = 0$ es sin pérdida; en imágenes con regiones planas grandes la compresión es muy alta (CR = 50 sin pérdida en «regions» 256^2; 21546 en «uniform»); los tiempos de construcción son de milisegundos; la recursión usa pila O(log n); τ es un control directo del compromiso tasa–distorsión.
- **Limitaciones.** La partición es rígida (por mitades), así que bordes diagonales o curvos obligan a subdividir hasta el píxel (círculo en «regions», Fig. 4); con ruido o texturas y τ pequeño el árbol **expande** los datos (hasta +5.6 %); el peor caso con «range» es $\Theta(n^2 \log n)$ en el modelo de lectura de píxeles; el color medio por bloque produce artefactos de bloques (Fig. 4) y el error por región es acotado, no el error global perceptual.
- **Cuándo conviene.** Gráficos, diagramas, logotipos, mapas, máscaras y píxel-art (regiones planas); también como estructura jerárquica para búsqueda espacial. No conviene en fotografías ruidosas con $\tau \approx 0$. Mejoras posibles: usar «var» o tablas de rango (O(1)) para la verificación, poda posterior de hijos similares, codificación entrópica de colores y partición adaptativa. **Limitaciones del estudio:** imágenes sintéticas, una sola máquina y tiempos de Python dominados por el costo por nodo.

Referencias y bibliotecas

[1] R. A. Finkel y J. L. Bentley, «Quad trees: a data structure for retrieval on composite keys», *Acta Informatica* 4(1), 1–9, 1974.

[2] H. Samet, «The Quadtree and Related Hierarchical Data Structures», *ACM Computing Surveys* 16(2), 187–260, 1984.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.ª/4.ª ed., MIT Press (Divide y Vencerás, Teorema Maestro).

[4] Z. Wang, A. C. Bovik, H. R. Sheikh y E. P. Simoncelli, «Image quality assessment: from error visibility to structural similarity», *IEEE Trans. Image Processing* 13(4), 600–612, 2004 (MSE/PSNR como métricas de referencia).

[5] F. C. Crow, «Summed-area tables for texture mapping», *SIGGRAPH Computer Graphics* 18(3), 207–212, 1984 (imágenes integrales).

Bibliotecas auxiliares: NumPy (matrices), Pillow (lectura/escritura de imágenes), Matplotlib (gráficas), pandas (CSV), ReportLab (este PDF). Ninguna implementa Quadtrees ni compresión: la construcción, recorrido, serialización y métricas son código propio del equipo.